

SIMATS ENGINEERING

ASSESSMENT TOOL 3

Name : SHEHSHATHRI

Reg No : 1925260316

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to

solve problems effectively. (BL3)

Assessment Tool Weightage: 30%

Duration : 45 Minutes

QUESTIONS: 2 Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Descriptive Written Test

QUESTION NO:

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

A **smart home automation system** is an AI-based system that observes the environment through sensors and takes suitable actions automatically. It may receive inputs such as **motion detection, temperature, time of day, door status, and user preferences**. An intelligent agent uses these percepts to decide what action should be performed.

The four important types of intelligent agents are **Simple Reflex Agent, Model-Based Agent, Goal-Based Agent, and Utility-Based Agent**.

1. Simple Reflex Agent

A **simple reflex agent** selects an action based only on the current percept. It follows predefined **condition-action rules**.

Example in a smart home:

- If motion is detected → turn ON the light.
- If temperature is above a specified value → turn ON the fan/air conditioner.
- If it is night and movement is detected outside → turn ON security lights.

Working:

Sensor input → Condition → Rule → Action

The main advantage is that it is **simple and fast**. However, it does not remember previous situations or consider future consequences. Therefore, it is less suitable for complex smart-home decisions.

2. Model-Based Agent

A **model-based agent** maintains an internal representation or model of the environment. It can use the current percept together with information about the previous state.

For example, if the system knows that:

- the user is currently away,
- the front door was opened,
- motion is detected inside the house,

it can understand the situation better than a simple reflex agent.

Example:

If the homeowner normally leaves the house at 8:00 AM and the security system detects movement at 10:00 AM while the house is marked as empty, the agent can identify this as an unusual situation and activate a security response.

The model-based agent is therefore useful for **security and situations where the current sensor input alone is not sufficient**.

3. Goal-Based Agent

A **goal-based agent** chooses actions according to a desired goal. Instead of simply responding to one sensor input, it considers which action will help achieve the goal.

Possible smart-home goals include:

- Maintain comfortable temperature.
- Reduce unnecessary electricity consumption.
- Keep the house secure.
- Automatically switch off unused devices.

Example:

Suppose the goal is **energy conservation**. If nobody is present in the living room, the agent may turn off the lights and air conditioner instead of keeping them ON.

Similarly, if the goal is **home security**, it may check door status, motion sensors, and time of day before deciding whether to activate an alarm.

A goal-based agent is more flexible than a simple reflex agent because it considers the desired outcome.

4. Utility-Based Agent

A **utility-based agent** chooses an action by considering how desirable the result will be. It evaluates different possible actions according to a **utility or preference measure**.

In a smart home, several objectives may conflict. For example:

- Maximum comfort may require more air-conditioning.
- Maximum energy efficiency may require less air-conditioning.
- Maximum security may require additional lighting and monitoring.

A utility-based agent can balance these factors.

Example:

Suppose the temperature is slightly high. Instead of immediately running the air conditioner at maximum power, the agent can consider:

- user's comfort,
- electricity consumption,
- current temperature,

- time of day,
- whether anyone is present.

It can then select an action that provides a good balance between **comfort and energy efficiency**.

Comparison of Agents in the Smart Home

Agent Type	How it Works	Smart Home Application	Limitation
Simple Reflex	Uses current percept and rules	Motion-based lighting	No memory or future consideration
Model-Based	Uses current input and internal state	Security and occupancy monitoring	More complex than reflex agent
Goal-Based	Selects actions to achieve goals	Energy saving and temperature control	Requires goals and decision-making
Utility-Based	Selects the most desirable action using utility	Balancing comfort, cost and security	More computationally complex

Most Effective Approach

For a smart home, a **hybrid approach combining model-based, goal-based, and utility-based agents with simple reflex rules** would be most effective.

Simple reflex rules can handle immediate actions such as turning ON a light when motion is detected. A model-based component can maintain information about **occupancy, previous conditions, and security state**. A goal-based component can work toward goals such as maintaining comfortable temperature and reducing energy consumption. Finally, a utility-based component can balance competing requirements such as **comfort, energy efficiency, and security**.

For example, when a person enters a room at night, the system can detect motion and turn on the light immediately. It can also check the person's normal usage pattern, current temperature, and energy-saving goal before deciding whether other devices should operate.

Conclusion

Therefore, a **hybrid intelligent-agent architecture** is suitable for a smart home because different agent types can perform different responsibilities. Simple reflex behavior provides fast responses, model-based reasoning provides environmental awareness, goal-based reasoning supports desired outcomes, and utility-based reasoning helps balance **comfort, energy efficiency, and security**. This combination provides a more intelligent and practical automation system.

QUESTION NO:

2. A robot is placed in an unknown maze and must find a path to the exit without prior

knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

Discuss the advantages and disadvantages of these algorithms in terms of time and

space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of

agent type and search strategy would be most effective and justify your choice.

A robot placed in an **unknown maze** does not initially know the complete layout of the environment. It has to explore the maze, obtain information through its sensors, and search for a path from its starting position to the exit.

Since there is no prior information about which path is better, **uninformed search strategies** such as **Uniform Cost Search (UCS)** and **Iterative Deepening Search (IDS)** can be used.

1. Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the **lowest path cost** first. It uses the accumulated cost from the starting point to the current state.

For a maze:

Start → possible paths → calculate path cost → expand lowest-cost path → continue until exit

If all movements have the same cost, UCS effectively searches for a shortest path in terms of the number of moves.

Example

Suppose the robot has three possible paths:

- Path A: cost = 8
- Path B: cost = 4
- Path C: cost = 6

UCS will explore the path with cost **4** first.

If different movements have different costs, UCS can select a path with a lower total cost even if it contains more steps.

Advantages of UCS

1. It can find the **least-cost solution** when step costs are appropriate.
2. It is useful when different movements have different costs.
3. It is complete under the usual positive-cost assumptions.
4. It considers the accumulated path cost before expanding a node.

Disadvantages of UCS

1. It can use a large amount of memory because it stores frontier nodes.
2. It may explore many unnecessary paths.

3. It can be slower when there are many possible paths.
4. Its performance is affected by the number of states and path costs.

Complexity

Let:

- b = branching factor
- C^* = cost of the optimal solution
- ϵ = minimum positive step cost

The time and space requirements can become very large, commonly expressed in exponential terms related to the cost of the optimal solution.

2. Iterative Deepening Search (IDS)

Iterative Deepening Search repeatedly performs depth-limited depth-first searches with increasing depth limits.

For example:

- First search: depth limit = 0
- Second search: depth limit = 1
- Third search: depth limit = 2
- Continue until the exit is found.

The process can be represented as:

Depth 0 → Depth 1 → Depth 2 → Depth 3 → ... → Goal

IDS combines some advantages of **Breadth-First Search** and **Depth-First Search**.

Example

If the exit is at depth 4, IDS searches:

0 → 1 → 2 → 3 → 4

When the search reaches the correct depth containing the exit, the robot finds the solution.

Advantages of IDS

1. It requires much less memory than breadth-first-style searches.
2. It is complete when the branching factor is finite.
3. With equal step costs, it can find a shallowest solution.
4. It is suitable when the depth of the solution is unknown.
5. It does not need prior knowledge of the maze.

Disadvantages of IDS

1. It repeatedly explores nodes from previous iterations.
2. This repeated searching increases execution time.
3. It is not suitable for finding a minimum-cost solution when step costs differ.
4. Its performance depends on the branching factor.

Complexity

For branching factor **b** and solution depth **d**:

- **Time complexity:** approximately $O(b^d)$
- **Space complexity:** approximately $O(bd)$

Thus, IDS has a major advantage in **space usage**, although it may perform repeated work.

UCS vs IDS

Feature	UCS	IDS
Search basis	Lowest path cost	Increasing depth limit
Complete	Yes, under positive step-cost assumptions	Yes, with finite branching
Optimal	Yes, for appropriate positive step costs	Yes for equal step costs
Memory	High	Low
Time	Can be high	Can be high due to

Feature	UCS	IDS
		repeated searches
Different path costs	Handles them	Does not directly optimize cost
Suitable for unknown solution depth	Yes	Yes
Main advantage	Least-cost path	Low memory requirement

Relationship with Intelligent Agents

The robot can also be considered an **intelligent agent** because it perceives the maze through sensors and takes actions such as moving forward, turning left, turning right, or moving backward.

Simple Reflex Agent

A simple reflex robot reacts to the current sensor input.

For example:

- If there is a wall in front → turn.
- If the path is clear → move forward.

This is fast but may not be able to remember visited locations. Therefore, it may repeatedly enter the same areas of the maze.

Model-Based Agent

A model-based agent maintains an internal representation of the maze.

It can remember:

- locations already visited,
- walls discovered,
- possible routes,
- previous actions.

This allows better decision-making in an unknown environment.

Goal-Based Agent

A goal-based robot has the goal of **reaching the maze exit**. It evaluates possible actions according to whether they help reach the goal.

The search algorithm can be used to identify a sequence of actions leading from the starting position to the exit.

Utility-Based Agent

A utility-based robot considers the quality of different solutions.

For example, it may consider:

- distance travelled,
- energy consumed,
- time taken,
- movement cost,
- safety of the route.

It can therefore choose a route that provides a better overall outcome rather than simply reaching the exit.

How Agent Design Affects Decision-Making

The design of the agent directly affects how the robot responds to the maze.

A **simple reflex agent** makes quick decisions based on immediate information but has limited memory. A **model-based agent** can use its internal state to make decisions based on information collected earlier. A **goal-based agent** focuses on reaching the exit, while a **utility-based agent** can compare different possible solutions and consider factors such as distance, time, and energy.

Therefore, as the agent design becomes more capable, the robot can use more information when making decisions.

Most Effective Combination

For an unknown maze, a **model-based, goal-based agent combined with IDS** can be an effective choice when **memory availability is limited and movement costs are equal**.

The **model-based component** allows the robot to maintain information about the maze as it explores. The **goal-based component** keeps the main objective of reaching the exit. IDS is useful because the robot does not know the depth of the solution in advance and IDS uses relatively low memory while gradually increasing the search depth.

However, if the maze has **different movement costs**—for example, some paths consume more energy or take more time—then **UCS with a model-based/goal-based agent** is more appropriate because UCS can search according to accumulated path cost.

Final Justification

Thus, the choice depends on the characteristics of the maze:

- **Equal movement costs + limited memory → Model-based/goal-based agent + IDS**
- **Different movement costs → Model-based/goal-based agent + UCS**

For a general unknown maze where memory efficiency is important and each movement has approximately equal cost, **IDS with a model-based goal-oriented agent** provides a suitable combination. If minimizing actual path cost is the main requirement, **UCS** should be preferred.

Conclusion

UCS and IDS are useful uninformed search strategies for solving an unknown maze. UCS focuses on the **lowest accumulated path cost**, while IDS gradually increases the search depth and uses considerably less memory. The intelligent-agent design determines how the robot perceives information, stores knowledge, defines its goal, and makes decisions. Combining an appropriate agent design with the suitable search strategy allows the robot to solve the maze effectively.